Now that we know how to specify conditions, let's turn our attention to the various methods supported by `ActiveRecord::Relation`, starting with `first` and `all`.

As you may have guessed, `first` returns the first row in the relation. It returns `nil` if the relation is empty. Similarly, `all` returns all the rows as an array. `ActiveRecord::Relation` also supports many of the methods of `Array` objects, such as `each` and `map`. It does so by implicitly calling the `all` first.

It's important to understand that the query isn't evaluated until one of these methods is used. This enables us to modify the query in a number of ways, namely, by calling additional methods, prior to making this call. Let's look at these methods now.

order

SQL doesn't require rows to be returned in any particular order unless we explicitly add an `order by` clause to the query. The `order` method lets us specify the criteria we'd normally add after the `order by` keywords. For example, the following query would return all of Dave's orders, sorted first by payment type and then by shipping date (the latter in descending order):

```
orders = Order.where(name: 'Dave').
  order("pay_type, shipped_at DESC")
```

limit

We can limit the number of rows returned by calling the `limit` method. Generally when we use the limit method, we'll probably also want to specify the sort order to ensure consistent results. For example, the following returns the first ten matching orders:

```
orders = Order.where(name: 'Dave').
  order("pay_type, shipped_at DESC").
  limit(10)
```

offset